

MEMORIA SPRINT 1

Grupo 12.08 “LongNight”

Integrantes:

Jorge Carvajal Lara

Sergi Torres Albert

Brad Owen Valladares Quispe

Javier Perepérez Cardona

Índice:

1. Arquitectura de la aplicación.....	3
1.1. Descripción general de la arquitectura	3
1.2.2. Capa de lógica de negocio	4
1.2.3. Capa de acceso a datos.....	4
1.3. Clases principales por capa	4
1.4. Diagrama de la arquitectura	5
1.5. Justificación de la arquitectura elegida	6
2. Aplicación del Patrón Factory Method.....	7
2.1. Descripción del patrón Factory Method	7
2.3. Implementación del patrón	8
2.3.1. Clases involucradas	8
2.3.2. Fragmentos de código relevantes	9
2.4. Ubicación del patrón en la arquitectura	11
2.5. Integración con el resto del sistema	11
2.6. Ventajas obtenidas con su uso	12
3. Uso de IA Generativa (AI Usage Log).....	13
4. Decisiones de Diseño (ADR)	17
ADR-001: Uso de Supabase como plataforma de Backend y Base de Datos.....	17
ADR-002: Gestión del estado de autenticación en el Frontend	19
ADR-003: Uso del patrón Factory Method para la creación de eventos	21
ADR-004: Refactorización de la relación Usuario-Rol-Evento.....	23
5. Conclusiones.....	25
5.1. Resultados obtenidos	25
5.2. Posibles mejoras futuras	25

1. Arquitectura de la aplicación

1.1. Descripción general de la arquitectura

El proyecto implementa una Arquitectura Cliente-Servidor, estructurada para mantener una clara separación de responsabilidades. La interacción de los usuarios se produce en el lado de cliente (Frontend) desarrollado en React utilizando Vite como herramienta de construcción.

Por otro lado, el Servidor (Backend), denominado Votify.API, actúa de núcleo de la aplicación, exponiendo una API REST desarrollada con ASP.NET Core (C#).

Internamente, el backend adopta una Arquitectura en Capas (N-Tier Architecture) clásica, que fragmenta de manera modular el comportamiento del sistema en responsabilidades especializadas: Presentación (Controladores), Negocio (Servicios), Acceso a datos (Repositorios) y Entidades de Dominio (Modelos).

Finalmente, para la persistencia y la gestión de datos remotos, se ha integrado Supabase como plataforma de Backend as a Service (BaaS), hacia la cual el backend ejecuta de forma centralizada todas sus transacciones.

1.2. Capas de la arquitectura

1.2.1. Capa de presentación (Web / React)

La capa de presentación en Votify se compone de dos frentes bien diferenciados que se comunican entre sí para brindar la experiencia final al usuario:

Por un lado, el Frontend, construido como una Single Page Application (SPA) utilizando la librería React y empaquetado mediante **Vite**. Esta cara visible para el usuario se encarga de la interfaz gráfica, el enrutamiento (estructurado en archivos como App.jsx) y la recolección de los datos introducidos por el usuario (en pantallas interactivas como OrganizerDashboard.jsx o los distintos pasos del asistente de creación de eventos). Toda la interacción visual y la comunicación asíncrona hacia el backend está centralizada en módulos específicos de la API en el cliente (como orgDashboardApi.js).

Por otro lado, dentro del servidor Backend, se encuentra la primera puerta de enlace para las peticiones HTTP: los Controladores (ubicados en la carpeta Controllers). Clases como AuthController, VotacionController o OrgDashboardController actúan exclusivamente como receptores y directores de tráfico. Su responsabilidad en esta capa se limita a validar que las peticiones entrantes sean correctas (recibir JSON y transformarlo), redirigir la acción hacia los "Servicios" (la capa de negocio) pidiendo que la ejecuten y, finalmente, empaquetar de vuelta una respuesta HTTP estándar (como códigos 200 OK, 400 Bad Request, etc.) hacia el cliente de React. Esta separación estricta garantiza que en ninguna circunstancia se incluya la lógica de negocio pura en los elementos de infraestructura web.

1.2.2. Capa de lógica de negocio

Esta capa, centralizada principalmente en el espacio de nombres de `Votify.API.Services`, aloja el verdadero "cerebro" o núcleo de la aplicación. Su función es procesar las peticiones procedentes de los controladores, aplicar las reglas, validaciones estrictas del dominio (por ejemplo, validar flujos de publicación de eventos o permisos en el panel del administrador) y coordinar el almacenamiento o extracción de los datos hacia los repositorios.

Está diseñada siguiendo los principios SOLID, destacando la fuerte inversión de dependencias mediante el uso extensivo de interfaces (`IAuthService`, `IEventoService`, `IVotoService`, `IOrgDashboardService`, etc.). Las clases que implementan estas interfaces encapsulan la lógica evitando acoplamientos innecesarios. Sumado a esto, se apoya en el empleo de patrones de diseño creacionales que asisten a los servicios, tales como las clases `VotoPublicoFactory` y `VotoJuradoFactory` ubicadas en el paquete `Factories`, que se encargan de abstraer y centralizar la creación compleja de entidades dependiendo de los requerimientos y el rol de usuario involucrado en la votación.

1.2.3. Capa de acceso a datos

Aislada en la carpeta `Repositories`, esta capa funciona como una capa de abstracción entre el dominio de negocio y la base de datos de Supabase. Su único propósito es encargarse de la persistencia y recuperación de datos, librando a la lógica de negocio de estas responsabilidades de infraestructura.

Esta capa implementa el Patrón Repositorio, definiendo "contratos" a través de interfaces (como `IProyectoRepository`, `IVotoRepository`, `IUsuarioRepository` o `ICategoriaRepository`) e implementando dichas abstracciones (en `ProyectoRepository`, `VotoRepository`, etc.). Éstas utilizan internamente el cliente de Supabase (`Supabase.Client`) para realizar las sentencias específicas. Esto garantiza que, ante futuros cambios de base de datos o esquemas, la única capa afectada en el sistema será exclusivamente los propios repositorios y se evitan consultas desordenadas en otros lugares de la aplicación.

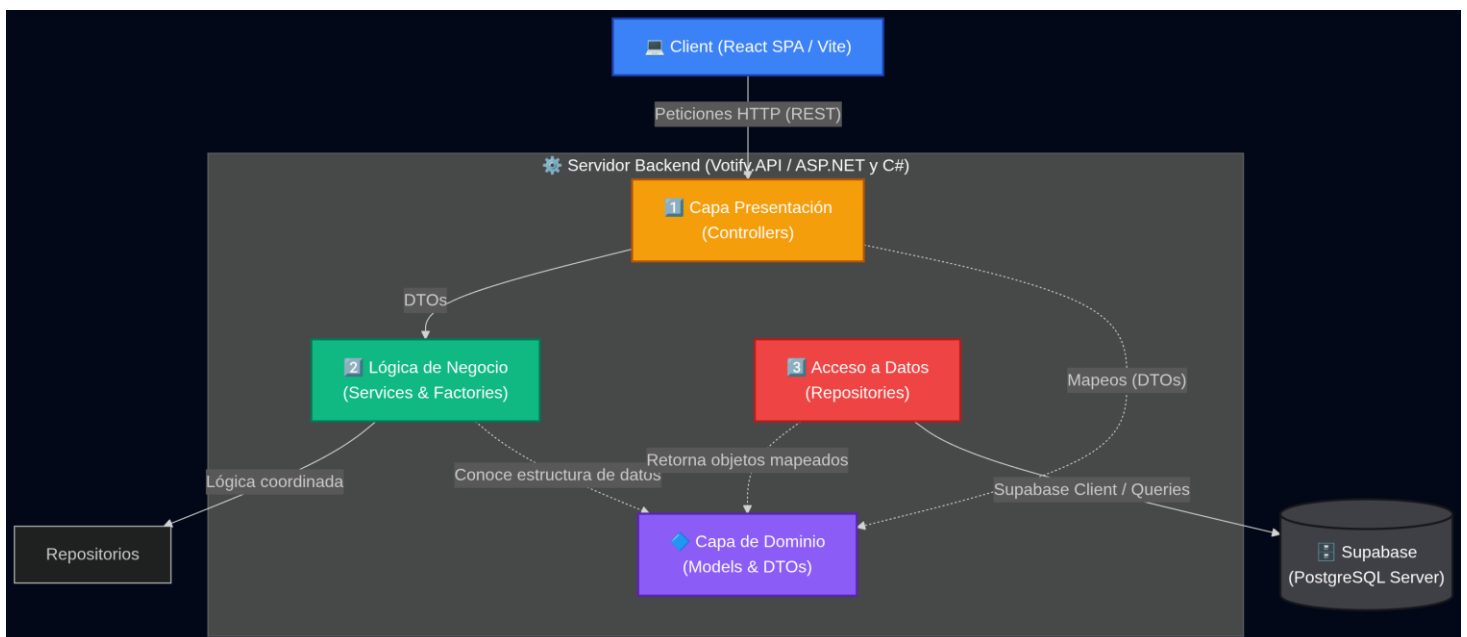
1.3. Clases principales por capa

A continuación se detallan las clases más significativas implicadas en el ecosistema, categorizadas según su respectiva capa:

- **Capa de Presentación (Controladores):**
 - `AuthController`: Punto de entrada para la orquestación del inicio de sesión y registro.
 - `OrgDashboardController`: Gestiona la interfaz y peticiones del panel de control de organizadores.

- CreateEventController y VotacionController: Expone rutas para los flujos de creación y emisión de votos.
- **Capa de Lógica de Negocio (Servicios y Factorías):**
 - CreateEventService: Gestiona la compleja y múltiple validación durante la creación de un nuevo evento.
 - VotoService: Orquesta la recepción, validación y cómputo de votos.
 - OrgDashboardService: Procesa las estadísticas en tiempo real y gestiona cierres o extensiones de votación.
 - VotoPublicoFactory / VotoJuradoFactory: Delegan la instanciación especializada de los tipos y pesos de votos.
- **Capa de Acceso a Datos (Repositorios):**
 - VotoRepository, ProyectoRepository, CategoriaRepository.
- **Capa de Dominio (Modelos):**
 - En Models/Domain: Contiene las clases que representan la lógica del negocio puro. Entre ellas destacan: Event con sus especificidades (SmallEvent, HackatonEvent), Usuario, Voto, Proyecto, Categoria. También contiene estructuras simplificadas bajo la jerarquía de DTOs para la transferencia de estados con la interfaz pública (Controller).

1.4. Diagrama de la arquitectura



1.5. Justificación de la arquitectura elegida

El enfoque arquitectónico planteado ha sido escogido por varias razones cruciales que garantizan el éxito en el desarrollo y mantenibilidad del sistema:

1. **Alta Cohesión y Bajo Acoplamiento (Separación de Responsabilidades):** En un proyecto de software, acoplar la toma de datos de base de datos a los controladores dificulta su tratamiento. Gracias a repartir la información en capas especializadas (Presentación, Lógica y Datos) permite modificar la implementación de cualquier capa (como una consulta a la BD o una regla de negocio) con el mínimo impacto colateral en las demás.
2. **Mantenibilidad y Trabajo en Paralelo:** Separar estricta y nativamente el Frontend del Backend favorece que múltiples desarrolladores trabajen concurrentemente. En el propio backend, al estar dividido en repositorios, modelos y servicios aislados por interfaces, previene conflictos o colisiones durante el desarrollo de módulos.
3. **Facilidad para el Testing:** La abstracción obligada dada por la Inyección de Dependencias a todos los Repositorios y Servicios a través sus interfaces, brinda la capacidad única de poder aislar el comportamiento de los objetos durante las pruebas (útil para los "Acceptance Tests" o Tests unitarios sin contactar al servidor o base de datos en tiempo real, inyectando simple y puramente versiones falsas o de mockeo).
4. **Optimización Estructural:** El uso de patrones como Factory Method provee en sistemas similares a este, la capacidad de lidiar de un modo elegante e instanciable a la gran cantidad y variedades de votos (Jurado, Público) dependientes del evento y del evaluador, retirándole a la lógica principal esa carga innecesaria.
5. **Uso de Supabase:** Utilizar un modelo Backend as a service facilita la concentración central del equipo sobre el dominio y las reglas lógicas (ASP.NET). Al delegar las configuraciones más tediosas, despliegue manual, control de concurrencia y roles de base de datos a un sistema en la nube rápido como Supabase.

2. Aplicación del Patrón Factory Method

2.1. Descripción del patrón Factory Method

El patrón Factory Method (o Método Fábrica) es una solución de diseño creacional cuya esencia es definir una interfaz o clase abstracta para la creación de un objeto, delegando en las subclases la decisión de qué clase concreta instanciar. En nuestro proyecto se aplica este método a la creación de tipos de eventos y de votos (EventCreator e IVotoFactory).

Al centralizar la creación en fábricas, liberamos a los servicios (como VotoService) de la responsabilidad de conocer los detalles de construcción, permitiéndoles enfocarse exclusivamente en la coordinación de la lógica de negocio.

2.2. Problema que resuelve en la aplicación

En la aplicación actualmente existen distintos tipos donde cada uno tiene características y lógica específica:

- Eventos (Hackathon, Innovation, SmallEvent)
- Votos (Público y Jurado)

Los problemas principales que teníamos sin el patrón son los siguientes:

- **Rigidez ante el cambio** (Violación del principio Open/Closed)
- **El Problema:** Antes, cada vez que quería añadir un nuevo tipo de voto (como un "Voto de Patrocinador") o un nuevo tipo de evento, tenía que abrir el VotoService o el CreateEventService y añadir más líneas a un if/else gigante.
- **La Solución:** Ahora el sistema está abierto a la extensión pero cerrado a la modificación. Para añadir un tipo nuevo, solo creas su clase y su fábrica. El VotoService y CreateEventService no se toca.

Acoplamiento fuerte (Dependencia de clases concretas)

- **El Problema:** Mi servicio "sabía" demasiado. Sabía que el VotoPublico no tenía evaluador y que el VotoJurado sí. Estaba atado a los detalles técnicos de cada clase.
- **La Solución:** Gracias al patrón, el servicio está desacoplado. Solo conoce la interfaz IVotoFactory y la clase abstracta Voto. No le importa qué pasa dentro de la creación; confía en la fábrica.

2.3. Implementación del patrón

2.3.1. Clases involucradas

Sobre los eventos

EventCreator (abstracto) define el método fábrica y sus tipos concretos son :

- SmallEventCreator
- HackatonCreator
- InnovationEventCreator

Implementan la creación concreta de eventos (clase evento) :

- HackatonEvent
- InnovationEvent
- SmallEvent

Sobre los votos

Factory:

- IVotoFactory

Implementaciones:

- VotoPublicoFactory
- VotoJuradoFactory

Productos (clase voto) concretos:

- VotoPublico
- VotoJurado

2.3.2. Fragmentos de código relevantes

Las fábricas:

```
4 references
public abstract class EventCreator
{
    4 references
    public abstract Event CreateEvent(
        int id,
        string nombre,
        string descripcion,
        DateTime fechaIni,
        DateTime fechaFin,
        string (parameter) int idorganizador
        int idorganizador,
        List<Categoria> categorias,
        List<Baremo> baremos,
        int codEvento);
}
```

```
3 references
public interface IVotoFactory
{
    // El Factory Method: devuelve el tipo base 'Voto'
    3 references
    Voto CrearVoto(int proyectoId, float valorBase, int idCategoria, int idCriterio, string? comentario, string? urlAudio, int? idUsuario, string? urlImagen);
}
```

Ejemplos de fabricas concretas:

```
1 reference
public class HackatonEventCreator : EventCreator
{
    2 references
    public override Event CreateEvent(
        int id,
        string nombre,
        string descripcion,
        DateTime fechaini,
        DateTime fechafin,
        string estado,
        int idorganizador,
        List<Categoria> categorias,
        List<Baremo> baremos,
        int codEvento)
    {
        return new HackatonEvent(
            id,
            nombre,
            descripcion,
            fechaini,
            fechafin,
            estado,
            idorganizador,
            categorias,
            baremos,
            codEvento);
    }
}
```

```
2 references
public Voto CrearVoto(int proyectoId, float valorBase, int idCategoria, int idCriterio, string? comentario, string? urlAudio, int? idU
{
    return new VotoJurado
    {
        IdProyecto = proyectoId,
        Valor = valorBase,
        IdCategoria = idCategoria,
        IdCriterio = idCriterio,
        Comentario = comentario,
        UrlAudio = urlAudio,
        FechaVoto = DateTime.UtcNow,
        IdEvaluador = idUsuario, // El jurado sí usa el idUsuario
        IpDispositivo = ipDispositivo // El IdEvaluador se asignará en el Service tras validar la sesión
    };
}
```

Y la creación de un objeto en el service , por ejemplo voto:

```
var factory = await ObtenerFactoryPorRolAsync(request.EventoId, idUsuario);

// Crear voto usando el patrón Factory
var voto = factory.CrearVoto(
    proyectoId: request.ProyectoId,
    valorBase: 1.0f, // Valor por defecto para público
    idCategoria: request.CategoriaId,
    idCriterio: 1, // TODO: Esto en el futuro seguramente será una lista de criterios
    comentario: request.Comentario,
    urlAudio: null,
    idUsuario: idUsuario,
    ipDispositivo: "web" // TODO: Obtener IP real
);

var votoCreado = await _votoRepository.AgregarVotoAsync(voto);
```

2.4. Ubicación del patrón en la arquitectura

Está ubicada en la carpeta de Backend/Votify.API/Factories como la lógica de su creación, e interacciona con Domain y Services.

El esquema de la arquitectura es:

Controller -> Service -> Factory -> Domain

2.5. Integración con el resto del sistema

El patrón se integra de la siguiente forma:

- Los Controllers reciben peticiones
- Los Services gestionan la lógica de negocio
- Los Factories crean los objetos adecuados
- Los Repositories/Supabase persisten los datos

Ejemplo con voto:

VotoController -> VotoService-> VotoFactory -> VotoPublico / VotoJurado-> Supabase

2.6. Ventajas obtenidas con su uso

Desacoplamiento (Independencia de clases concretas)

El patrón elimina la dependencia directa entre el cliente (por ejemplo, el VotoService) y las clases concretas (VotoJurado, VotoPublico). Al interactuar exclusivamente a través de interfaces (IVotoFactory) y clases abstractas (Voto), el núcleo de la aplicación no necesita conocer los detalles de implementación de cada objeto. Esto permite cambiar o evolucionar los tipos de votos y eventos sin que el resto del sistema se vea afectado.

Escalabilidad (Crecimiento seguro del sistema)

La escalabilidad en Votify se vuelve lineal y segura. Añadir un nuevo tipo de evento (ej. ESportsEvent) o un nuevo tipo de voto (ej. SponsorVote) no requiere modificar la lógica de negocio existente. Al no haber bloques switch o if/else dependientes del tipo, el riesgo de introducir errores en funcionalidades que ya funcionan (regresión) es prácticamente nulo. El sistema crece simplemente añadiendo nuevas piezas, no alterando las actuales.

Mantenibilidad (Centralización de la lógica)

Se evita la duplicación de la lógica de creación en distintas partes del proyecto. Al centralizar el "cómo" se construye cada objeto dentro de su propia fábrica concreta, cualquier cambio en los requisitos de instanciación (como añadir un nuevo parámetro obligatorio o cambiar una validación inicial) se realiza en un único punto. Esto facilita enormemente el mantenimiento a largo plazo y la corrección de errores.

Principio de Abierto/Cerrado (OCP)

Este es uno de los pilares de SOLID que el Factory Method cumple a la perfección. El sistema queda abierto a la extensión, permitiendo que los desarrolladores añadan nuevas categorías de votos o eventos en el futuro, pero permanece cerrado a la modificación, ya que el código que coordina las votaciones no necesita ser alterado para dar soporte a estas nuevas incorporaciones.

Código más limpio (Reducción de complejidad cognitiva)

La aplicación se libera de "código redundante" o bloques condicionales extensos que dificultan la lectura. Al delegar la responsabilidad de creación a las fábricas, el código del servicio se vuelve mucho más legible y elegante, centrándose exclusivamente en coordinar el flujo de la votación. Esto no solo mejora la estética del código, sino que facilita la entrada de nuevos desarrolladores al proyecto y mejora la calidad de las pruebas unitarias.

3. Uso de IA Generativa (AI Usage Log)

AI Usage Log — Sprint S1

1) Herramientas usadas

- Herramienta/modelo: Gemini, Claude, Copilot, Antigravity
- Para qué se usó:
 - Planificación y refinamiento de requisitos.
 - Arquitectura y diseño del sistema (Backend y Frontend).
 - Generación de lógica de negocio y código boilerplate.

2) Prompts clave (copiar/pegar) + enlace

Prompt 1:

Estoy implementando el Dashboard principal de eventos en React @Frontend\src\pages\DashboardPage.jsx. Necesito un componente funcional que cargue los eventos desde la API usando el token JWT guardado en localStorage y la ID del usuario. Debe mostrar una lista de componentes EventCard, incluir estado de carga y manejo de errores, y añadir un buscador que filtre los eventos por nombre. Además, quiero que el diseño use únicamente las variables CSS definidas en @Frontend\src\index.css y que siga la @Guia_Diseño.md.

(Sin enlace: Chat en Antigravity)

Prompt 2:

En el service de Event, necesito tener dos métodos que mapeen tanto los baremos como las categorías para que a la hora de crear dichos eventos, utilicen aquellos que se les pase en los dtos, dicho esto utiliza los dtos para la generación de los métodos y explícame posteriormente como enlazar el archivo CreateEventService con su correspondiente controller.

(Sin enlace: Chat en Antigravity)

Prompt 3:

Vale mejor para que sea más óptimo créame en CategoríasRepository una query que obtenga las categorías del eventold, también hazme uno para traer solo los proyectos de cada categoría que tengamos, y después cambia el service.

(Sin enlace: Chat en Copilot)

Prompt 4:

1. Estoy trabajando en un proyecto de ingeniería informática en el que hay una backend y una frontend. He añadido una clase a la backend en la carpeta Domain y la base de datos Supabase que tenemos como "ComentarioCualitativo". Pero luego, también en el backend hay otra carpeta con DTOs y luego en el frontend con APIs. ¿Me puedes explicar qué son y qué aporta cada cosa, junto al propio Domain? Y de paso cómo almacenar y hacer consultas a la base de datos?
2. Muéstrame cómo hacer un endpoint completo (DTO → insert → respuesta).
3. Adjunto el código de VotosPage. Ya está conectada al proyecto, pero necesito cargar de la API los comentarios dados ciertos ids de votación. ¿Cómo lo hago? En particular quiero que se muestren en "MOCK DATA". ¿Cómo uso la API dada para cargarlos? Si tengo que modificar algo dímelo.
4. Me da problemas, no me reconoce el "Ok" ni el "Request", ¿hay algún using que no esté usando? El problema es que en "Ok" y en "Request" me da errores, y estoy usando los using.

(Sin enlace: Chat en Copilot)

3) Salidas relevantes (resumen corto)

Prompt 1:

- Propuso un componente funcional en React para la carga de eventos mediante useEffect, manejando el estado de carga, errores y persistencia local inicial.
- Sugirió el uso de useMemo para el filtrado eficiente por nombre en el cliente.

Prompt 2:

- Generó los métodos MapearBaremos y MapearCategorias en C#.
- En MapearCategorias, implementó bucles anidados para gestionar la jerarquía de categorías y pesos de forma relacional.

Prompt 3:

- Creó las queries específicas en los repositorios de C# y adaptó mostrarDashboardAsync para usarlas.
- Propuso una lógica de refresco de datos agresiva (llamadas constantes a BD).

Prompt 4:

- Explicó la arquitectura de capas (Domain, DTO, API) y su propósito en la separación de responsabilidades.

- Generó el código base para el controlador, DTOs y la integración con la API del frontend para los comentarios cualitativos.

4) Qué aceptamos y qué rechazamos

- Prompt 1 Aceptado: Se aceptó íntegramente ya que la solución estaba alineada con las buenas prácticas de React y respetaba estrictamente la Guia_Diseño.md y las variables CSS del proyecto.
- Prompt 2 Aceptado: Los métodos de mapeo eran lógicos, resolvían la complejidad de la estructura de datos y funcionaron correctamente tras las pruebas manuales.
- Prompt 3 Corregido: Se aceptó la lógica de las queries en los repositorios, pero se rechazó el borrado del método que gestionaba el refresco de datos. Se corrigió para que no hiciera llamadas innecesarias a la BD una vez iniciada la votación, ya que las categorías son estáticas en esa fase.
- Prompt 4 Corregido: Se aceptó la estructura base y los DTOs, pero se rechazaron detalles de implementación del controlador que daban errores de compilación (usings faltantes o métodos mal nombrados) y se realizó un control humano exhaustivo para ajustar la lógica a la arquitectura del proyecto.

5) Cómo lo verificamos

- Tests unitarios (indicar cuáles)
- Experimento / medición: Pruebas manuales de carga de eventos, creación y visualización de comentarios en el Dashboard.
- Consulta a la documentación oficial: Revisión de la documentación de Supabase C# Client y React hooks.
- Revisión por pares: Pulido iterativo del código del frontend y backend para asegurar coherencia.

6) Resultado final / decisión humana

- Prompt 1: Implementado y fusionado. Reflejado en la carga principal de eventos del Dashboard.

Commit: 33db86c

- Prompt 2: Solución aplicada al servicio de creación de eventos.

Commit: bb86f63

- Prompt 3: Implementado tras corregir la lógica de refresco para optimizar el rendimiento.

Commit: 5b3bfcc

- Prompt 4: La base generada permitió estructurar la funcionalidad de comentarios cualitativos, aunque requirió ajustes manuales significativos.

Commit: 7d7d7b3

4. Decisiones de Diseño (ADR)

ADR-001: Uso de Supabase como plataforma de Backend y Base de Datos

Fecha: 10-03-2026

Sprint: S1

Estado: Aprobado

1) Contexto

El proyecto requiere una infraestructura de backend robusta que gestione la persistencia de datos (PostgreSQL), la autenticación de usuarios (JWT, gestión de sesiones) y funcionalidades en tiempo real. En lugar de gestionar manualmente servidores de bases de datos y servicios de autenticación complejos (como Identity en .NET Core o servidores OAuth propios), se busca una solución que acelere el desarrollo y reduzca la carga operativa de mantenimiento.

2) Opciones consideradas

- **Opción A: Supabase.** Utilizar Supabase como plataforma integral que proporciona PostgreSQL, Auth, Storage y Realtime de forma gestionada.
- **Opción B: Base de datos SQL tradicional (PostgreSQL/SQL Server) gestionada localmente o en VPS.** Configurar y mantener manualmente la base de datos y desarrollar toda la lógica de autenticación desde cero en el Backend.
- **Opción C: Firebase (BaaS).** Similar a Supabase pero basado en NoSQL (Firestore).

3) Criterios de decisión

- **Velocidad de desarrollo:** Facilidad para integrar autenticación y base de datos sin configuración extensa.
- **Relacionalidad:** El dominio de Votify es relacional, por lo que se prefiere SQL sobre NoSQL.
- **Costo operativo:** Minimizar la necesidad de administrar servidores y actualizaciones de seguridad.
- **Ecosistema y SDK:** Disponibilidad de librerías oficiales y mantenidas para .NET y React.
- **Tiempo real:** Capacidad nativa para notificaciones de votos o cambios en el ranking.

4) Decisión tomada

Se elige la **Opción A (Supabase)**. Supabase ofrece la potencia de PostgreSQL (necesaria para el modelo relacional de eventos, proyectos y votos) con la simplicidad de un servicio en la nube gestionado. Su sistema de autenticación integrado simplifica enormemente la gestión de usuarios y roles. Además, el uso de su SDK oficial tanto en el Backend (C#) como en el Frontend (React) permite una integración coherente y tipada mediante atributos y modelos base.

La Opción B se descartó por el alto esfuerzo de mantenimiento y desarrollo de seguridad. La Opción C se descartó porque el modelo de datos relacional de Votify encaja mejor en PostgreSQL que en Firestore.

5) Consecuencias

Positivas:

- Reducción drástica del tiempo de implementación de la capa de datos y seguridad.
- Escalabilidad automática y backups gestionados.

Negativas / trade-offs:

- Dependencia de un proveedor externo.
- Límites de uso en la capa gratuita (aunque suficientes para el alcance actual del proyecto).

Riesgos y mitigaciones:

- Interrupciones del servicio externo: mitigado con la posibilidad teórica de exportar el esquema y datos de PostgreSQL a cualquier otro hosting SQL, dado que Supabase es código abierto.

6) Evidencia

- Integración en Backend/Votify.API/Program.cs usando Supabase.Client.
- Modelos de dominio heredando de Supabase.Postgrest.Models.BaseModel.
- Configuración de variables de entorno SUPABASE_URL y SUPABASE_KEY en .env.
- Dependencia @supabase/supabase-js en el package.json del Frontend.

ADR-002: Gestión del estado de autenticación en el Frontend

Fecha: 13-04-2026

Sprint: S1

Estado: Aprobado

1) Contexto

La aplicación web de Votify necesita una forma consistente de manejar el estado de autenticación del usuario (tokens de acceso, identificación del usuario y nombre) a través de múltiples componentes. Es crucial que la información de sesión sea persistente y reactiva.

2) Opciones consideradas

- **Opción A: React Context API.** Crear un proveedor de contexto global que envuelva la aplicación y exponga el estado y las funciones de autenticación.
- **Opción B: Redux / Zustand.** Utilizar una librería externa de gestión de estado global para manejar la autenticación.
- **Opción C: Props Drilling.** Pasar el estado de autenticación manualmente desde el componente raíz a todos los subcomponentes que lo necesiten.
- **Opción D: Variables globales fuera del ciclo de vida de React.** Almacenar el token en una variable estática o global, accediendo directamente a `localStorage` en cada componente.

3) Criterios de decisión

- **Simplicidad:** Minimizar el uso de librerías externas innecesarias para una tarea puntual.
- **Reactividad:** Los componentes deben reaccionar automáticamente a los cambios de estado.
- **Escalabilidad:** Debe ser fácil de usar en toda la aplicación sin generar un código difícil de mantener.
- **Persistencia:** Capacidad de mantener la sesión entre recargas del navegador.

4) Decisión tomada

Se elige la **Opción A (React Context API)**. Es la solución estándar e integrada de React para compartir estado global simple como la autenticación. Evita la complejidad y el peso de librerías como Redux, siendo suficiente para las necesidades actuales de

Votify. Al combinar el Contexto con `localStorage` y el hook `useEffect`, se logra tanto la reactividad como la persistencia de forma nativa.

La Opción C se descartó por la alta profundidad del árbol de componentes, y la Opción D se descartó por la falta de reactividad que obligaría a recargas manuales o lógica propensa a errores.

5) Consecuencias

Positivas:

- Acceso sencillo al estado de autenticación desde cualquier componente mediante hooks (`useAuth`).
- No se requieren dependencias externas adicionales.
- Sincronización automática entre pestañas mediante el evento `storage`.

Negativas / trade-offs:

- Posibles re-renderizados innecesarios si el contexto se vuelve muy grande (mitigado manteniendo el contexto enfocado solo en autenticación).
- Mayor acoplamiento de los componentes al proveedor de contexto.

Riesgos y mitigaciones:

- Almacenamiento de tokens en `localStorage`: mitigado con buenas prácticas de seguridad en el backend y limpieza de entradas en el frontend.

6) Evidencia

- Implementación en `Frontend/src/context/AuthContext.jsx`.
- Uso mediante hook personalizado en `Frontend/src/hooks/useAuth.js`.
- Protección de rutas en `Frontend/src/components/auth/ProtectedRoute.jsx`.

ADR-003: Uso del patrón Factory Method para la creación de eventos

Fecha: 21-03-2026

Sprint: S1

Estado: Aprobado

1) Contexto

El sistema Votify debe soportar múltiples tipos de eventos (Hackaton, Innovation Fair, Small Events), cada uno con sus propias reglas de negocio, validaciones y estructuras de datos específicas (aunque compartan una base común). La creación directa de estos objetos en los servicios de aplicación (como `CreateEventService`) generaría un acoplamiento fuerte y violaría el principio de Responsabilidad Única (SRP), dificultando la extensión del sistema cuando se requieran nuevos tipos de eventos en el futuro.

2) Opciones consideradas

- **Opción A: Implementar el patrón Factory Method.** Definir una clase abstracta creadora con un método de fábrica que las subclases concretas implementen para instanciar el tipo de evento específico.
- **Opción B: Uso de lógica condicional (switch/if) en el servicio.** Manejar la instanciación dentro de un método en el servicio principal basándose en un discriminador de tipo de evento.
- **Opción C: Patrón Abstract Factory.** Crear una interfaz para familias de objetos relacionados (por ejemplo, si cada evento también tuviera una familia de reportes o validadores específicos).

3) Criterios de decisión

- **Extensibilidad:** Facilidad para añadir nuevos tipos de eventos sin modificar el código existente (Principio Open/Closed).
- **Desacoplamiento:** El código que solicita la creación no debe conocer las clases concretas de los eventos.
- **Mantenibilidad:** Centralizar la lógica de instanciación para evitar duplicidad y facilitar cambios.
- **Simplicidad:** No sobre-diseñar si los requerimientos no lo exigen.

4) Decisión tomada

Se elige la **Opción A (Factory Method)**. Esta decisión permite que el sistema sea altamente extensible. Al delegar la responsabilidad de la instanciación a clases

específicas (`HackatonEventCreator`, `InnovationFairEventCreator`, `SmallEventCreator`), el servicio de creación solo interactúa con la abstracción `EventCreator`. Esto cumple con el principio de Inversión de Dependencia y permite que añadir un nuevo tipo de evento solo requiera crear una nueva clase `Event` y su correspondiente `EventCreator`.

La Opción C se descartó por ser demasiado compleja para el estado actual del proyecto, y la Opción B se descartó por violar el principio Open/Closed y dificultar el mantenimiento a largo plazo.

5) Consecuencias

Positivas:

- Cumplimiento de los principios SOLID.
- Facilita las pruebas unitarias al poder mockear los creadores.
- Código más limpio y organizado por responsabilidades.

Negativas / trade-offs:

- Mayor número de clases en el proyecto (una clase creadora por cada tipo de evento).
- Ligero incremento en la complejidad inicial del diseño.

Riesgos y mitigaciones:

- Riesgo de "explosión de clases": mitigado manteniendo la jerarquía clara y solo creando subclases cuando realmente existan diferencias de comportamiento o estructura.

6) Evidencia

- Estructura de archivos en `Backend/Votify.API/Factories/`: `EventCreator.cs`, `HackatonEventCreator.cs`, etc.
- Implementación de modelos en `Backend/Votify.API/Models/Domain/`: `Event.cs`, `HackatonEvent.cs`, `InnovationFairEvent.cs`, `SmallEvent.cs`.

ADR-004: Refactorización de la relación Usuario-Rol-Evento

Fecha: 23-03-2026

Sprint: S1

Estado: Aprobado

1) Contexto

Actualmente, el rol de un usuario (Organizador, Jurado, Participante) está definido directamente en la clase Usuario. Sin embargo, un usuario puede tener diferentes roles según el evento en el que participa. Esto genera una limitación funcional y de modelado, ya que no se puede representar correctamente la relación muchos a muchos entre usuarios y eventos con roles variables. Es necesario rediseñar el modelo para soportar esta flexibilidad y mantener la integridad de los datos, afectando tanto al modelo de dominio como a la base de datos y la lógica de acceso.

2) Opciones consideradas

- **Opción A:** Crear una entidad intermedia EventoUsuario que relacione Usuario y Evento, incluyendo el rol como atributo. Así, un usuario puede tener distintos roles en diferentes eventos.
- **Opción B:** Mantener el rol en Usuario y almacenar una lista de eventos por cada usuario, gestionando los roles como una colección de pares (evento, rol) en la propia entidad Usuario.
- **Opción C:** No hacer nada y mantener el modelo actual, asumiendo que un usuario solo puede tener un rol global en el sistema.

3) Criterios de decisión

- Mantenibilidad y claridad del modelo de datos
- Extensibilidad para futuros requisitos
- Normalización y consistencia en la base de datos
- Facilidad de consulta y manipulación desde los repositorios y controladores
- Impacto en la migración de datos y compatibilidad hacia atrás

4) Decisión tomada

Se elige la Opción A: crear una entidad intermedia EventoUsuario que represente la participación de un usuario en un evento con un rol específico. Esta opción normaliza la relación, permite añadir fácilmente nuevos atributos y facilita consultas eficientes. Requiere modificar el modelo de dominio en .NET, los repositorios y controladores para

gestionar la nueva entidad, así como crear la tabla intermedia correspondiente en la base de datos.

5) Consecuencias

Positivas:

- Permite que un usuario tenga diferentes roles según el evento.
- Facilita la extensión futura (más atributos en la relación, nuevos roles).
- Mejora la integridad y normalización de los datos.

Negativas / trade-offs:

- Incrementa la complejidad del modelo y de las consultas.
- Requiere migración de datos y refactorización de la lógica existente.

Riesgos y mitigaciones:

- Riesgo de errores en la migración: mitigado con pruebas y migraciones controladas.
- Posible aumento de la complejidad en los controladores: mitigado con servicios y repositorios bien definidos.

6) Evidencia

- Commit relevante: d9ba5a1 — fix: patrón diseño usuario y bugs visuales login.
- Base de datos actualizada con nueva tabla evento_usuario.

5. Conclusiones

5.1. Resultados obtenidos

Al finalizar el Sprint 1, se ha cumplido satisfactoriamente el objetivo general de desarrollar el ciclo completo de una votación de tipo anónimo, alcanzando los siguientes hitos:

1. Votación Popular Anónima: Se ha implementado un sistema que permite a cualquier usuario (público general) participar sin necesidad de registro previo, cumpliendo con la premisa de anonimato y permitiendo la selección múltiple de proyectos hasta un límite parametrizado.
2. Mecanismo de Feedback Cualitativo: Se integró una funcionalidad básica para que los votantes dejen comentarios dirigidos a los competidores, fomentando el intercambio de valor más allá de la puntuación numérica.
3. Visualización de Resultados y Ranking: Se desarrolló un Dashboard en tiempo real que procesa la contabilización de votos y presenta un ranking dinámico basado en la participación popular.
4. Consulta de Feedback para Competidores: Se habilitó una vista específica donde los participantes pueden visualizar de forma transparente los comentarios y valoraciones recibidas.
5. Infraestructura Base (UTs adicionales): Más allá de los mínimos requeridos, el equipo ha consolidado el sistema de autenticación para organizadores y el asistente de configuración de eventos, sentando las bases para la gestión profesional de futuras competiciones.

5.2. Posibles mejoras futuras

Para la siguiente fase del proyecto, el equipo se centrará en la profesionalización del sistema de evaluación y la automatización de la gestión, abordando las siguientes Unidades de Trabajo (UT) acordadas:

- Gestión Avanzada de Contenido: Implementación de la creación y gestión real de proyectos y la parametrización avanzada de límites de votos.
- Evolución del Sistema de Votación: Transición hacia modelos complejos con la inclusión de Jurado Autenticado y la gestión de puntuaciones por criterios específicos y comentarios detallados por cada criterio.

- **Control y Automatización:** Desarrollo de herramientas de control manual para el organizador sobre el estado de las votaciones, junto con la automatización de ventanas temporales y la configuración de reglas de negocio como la obligatoriedad de comentarios.
- **Seguridad y Auditoría:** Refuerzo de la integridad del proceso mediante el control de unicidad en la votación popular y la actualización selectiva de comentarios visibles.
- **Comunicación:** Integración de un sistema de notificaciones automáticas para mantener informados a todos los roles del evento sobre hitos críticos.